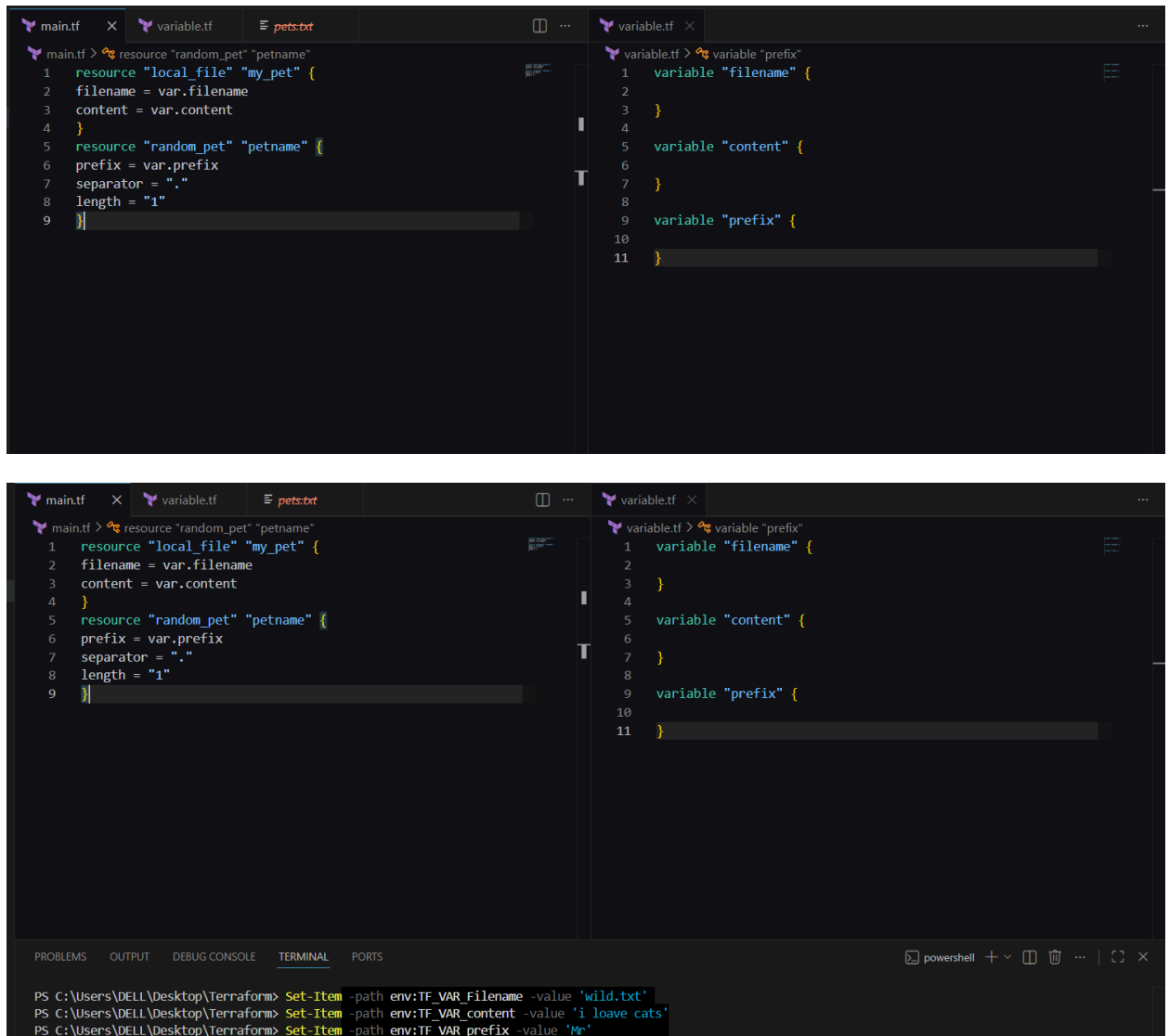


Terraform-03

1. Execute the script shown in the video.

Set environmental variables



The image shows a VS Code editor with two Terraform configuration files and a PowerShell terminal at the bottom.

main.tf

```
main.tf > resource "random_pet" "petname"
1  resource "local_file" "my_pet" {
2    filename = var.filename
3    content = var.content
4  }
5  resource "random_pet" "petname" {
6    prefix = var.prefix
7    separator = "."
8    length = "1"
9  }
```

variable.tf

```
variable.tf > variable "prefix"
1  variable "filename" {
2
3  }
4
5  variable "content" {
6
7  }
8
9  variable "prefix" {
10
11 }
```

Terminal

```
PS C:\Users\DELL\Desktop\Terraform> Set-Item -path env:TF_VAR_Filename -value 'wild.txt'
PS C:\Users\DELL\Desktop\Terraform> Set-Item -path env:TF_VAR_content -value 'i loave cats'
PS C:\Users\DELL\Desktop\Terraform> Set-Item -path env:TF_VAR_prefix -value 'Mr'
```

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell + - [ ] [ ] ... [ ] [ ] [ ] [ ]
Enter a value: yes

local_file.my_pet: Destroying... [id=d1f48b051f876761cdd2ab566ddc899593fafa85]
random_pet.pet: Destroying... [id=MR.measured.clam]
random_pet.pet: Destruction complete after 0s
random_pet.petname: Creating...
local_file.my_pet: Destruction complete after 0s
random_pet.petname: Creation complete after 0s [id=Mr.pug]
local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s [id=a867117774ad3d9862d1db44b6c72c3e27335d03]

Apply complete! Resources: 2 added, 0 changed, 2 destroyed.
PS C:\Users\DELL\Desktop\Terraform>

```

Resource dependency:

```
main.tf  X  ≡ pets.txt
```

```
main.tf > resource "local_file" "my_pet" > content
1  resource "local_file" "my_pet" {
2    filename = "pets.txt"
3    content = "my cat name is ${random_pet.petname.id}"
4  }
5  resource "random_pet" "petname" {
6    prefix = "MR"
7    separator = "."
8    length = "1"
9  }
10
```

```
main.tf x pets.txt
main.tf > resource "local_file" "my_pet" > content
1 resource "local_file" "my_pet" {
2   filename = "pets.txt"
3   content = "my cat name is ${random_pet.petname.id}"
4 }
5 resource "random_pet" "petname" {
6   prefix = "MR"
7   separator = "."
8   length = "1"
9 }
10

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\DELL\Desktop\Terraform> terraform apply
var.filename
Enter a value: yes

random_pet.petname: Refreshing state... [id=Mr.pug]
local_file.my_pet: Refreshing state... [id=a867117774ad3d9862d1db44b6c72c3e27335d03]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
-/+ destroy and then create replacement

Terraform will perform the following actions:

# local_file.my_pet must be replaced
-/+ resource "local_file" "my_pet" {
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Enter a value: yes

local_file.my_pet: Destroying... [id=a867117774ad3d9862d1db44b6c72c3e27335d03]
local_file.my_pet: Destruction complete after 0s
random_pet.petname: Destroying... [id=Mr.pug]
random_pet.petname: Destruction complete after 0s
random_pet.petname: Creating...
random_pet.petname: Creation complete after 0s [id=MR.platypus]
local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s [id=746db925e547a068deec611d64b7a64f1a7688fd]

Apply complete! Resources: 2 added, 0 changed, 2 destroyed.
PS C:\Users\DELL\Desktop\Terraform>
```

```
main.tf x pets.txt x
pets.txt
1 my cat name is MR.platypus
```

Output variables:

```
main.tf  X  pets.txt
main.tf > output "pet_name"
1  resource "local_file" "my_pet" {
2    filename = "pets.txt"
3    content = "my cat name is ${random_pet.petname.id}"
4  }
5  resource "random_pet" "petname" {
6    prefix = "MR"
7    separator = "."
8    length = "1"
9  }
10
11 output "pet_name" {
12   value = random_pet.petname.id
13 }
14
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\DELL\Desktop\Terraform> terraform apply
var.filename
  Enter a value: yes

random_pet.petname: Refreshing state... [id=MR.platypus]
local_file.my_pet: Refreshing state... [id=746db925e547a068deec611d64b7a64f1a7688fd]

Changes to Outputs:
  + pet_name = "MR.platypus"

You can apply this plan to save these new output values to the Terraform state, without changing any real infrastructure.

Do you want to perform these actions?
  Terraform will perform the actions described above.
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

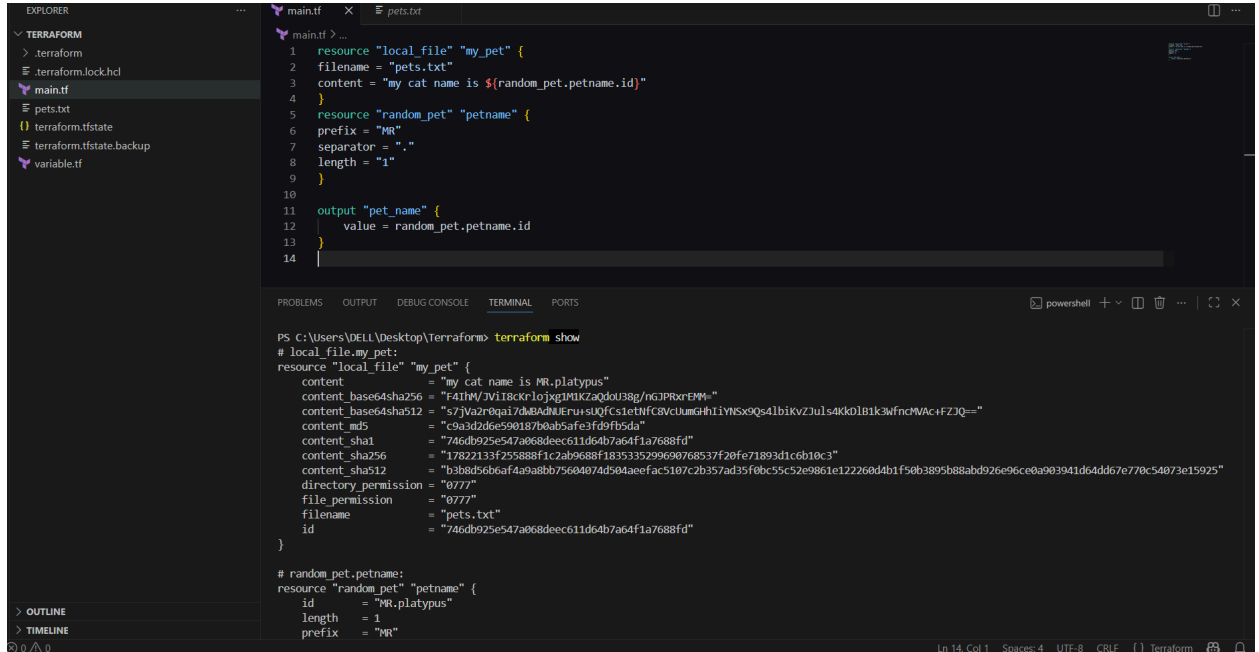
Outputs:

pet_name = "MR.platypus"
PS C:\Users\DELL\Desktop\Terraform>
```

We can use also like:

```
PS C:\Users\DELL\Desktop\Terraform> terraform output
pet_name = "MR.platypus"
PS C:\Users\DELL\Desktop\Terraform>
```

State file:

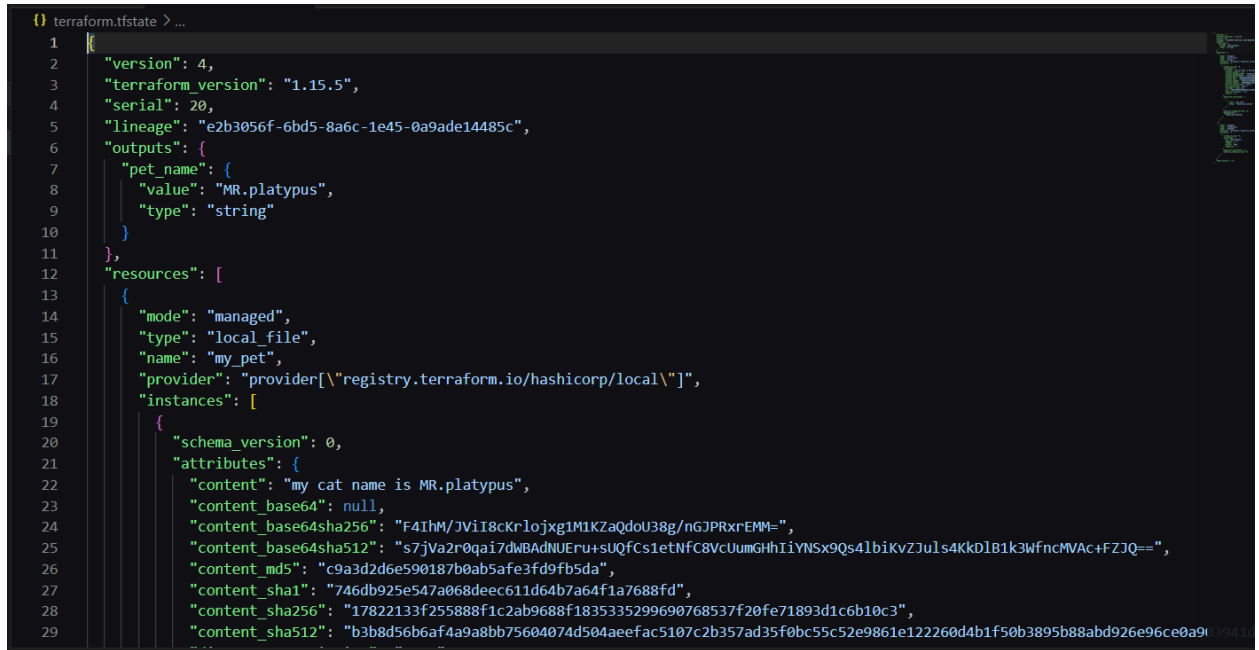


The screenshot shows a VS Code editor with a Terraform configuration file named `main.tf` and its state output. The configuration file contains two resources: `local_file` and `random_pet`. The state output shows the values of these resources after they have been applied.

```
main.tf > ...
1 resource "local_file" "my_pet" {
2   filename = "pets.txt"
3   content = "my cat name is ${random_pet.petname.id}"
4 }
5 resource "random_pet" "petname" {
6   prefix = "MR"
7   separator = "."
8   length = "1"
9 }
10
11 output "pet_name" {
12   value = random_pet.petname.id
13 }
14
```

```
PS C:\Users\DELL\Desktop\Terraform> terraform show
# local_file.my_pet:
resource "local_file" "my_pet" {
  content = "my cat name is MR.platypus"
  content_base64sha256 = "F4IHM/JViI8ckrlojxg1M1KZaQd0U38g/ngJPRxrEMM="
  content_base64sha512 = "s7jVa2r0qai7dwADNUERu+sUQfcs1etnFC8VcUumGHHiiYNSx9Qs4lBikvZJulS4KkDlB1k3WfncWVAc+FZJQ=="
  content_md5 = "c9a3d2d6e590187b0ab5afe3fd9fb5da"
  content_sha1 = "746db925e547a068deec611d64b7a64f1a7688fd"
  content_sha256 = "17822133f255888f1c2ab9688f1835335299690768537f20fe71893d1c6b10c3"
  content_sha512 = "b3b8d56b6af4a9a8bb75604074d504aeeefac5107c2b357ad35f0bc55c52e9861e122260d4b1f50b3895b88abd926e96ce0a90941d64dd67e770c54073e15925"
  directory_permission = "0777"
  file_permission = "0777"
  filename = "pets.txt"
  id = "746db925e547a068deec611d64b7a64f1a7688fd"
}

# random_pet.petname:
resource "random_pet" "petname" {
  id = "MR.platypus"
  length = 1
  prefix = "MR"
}
```



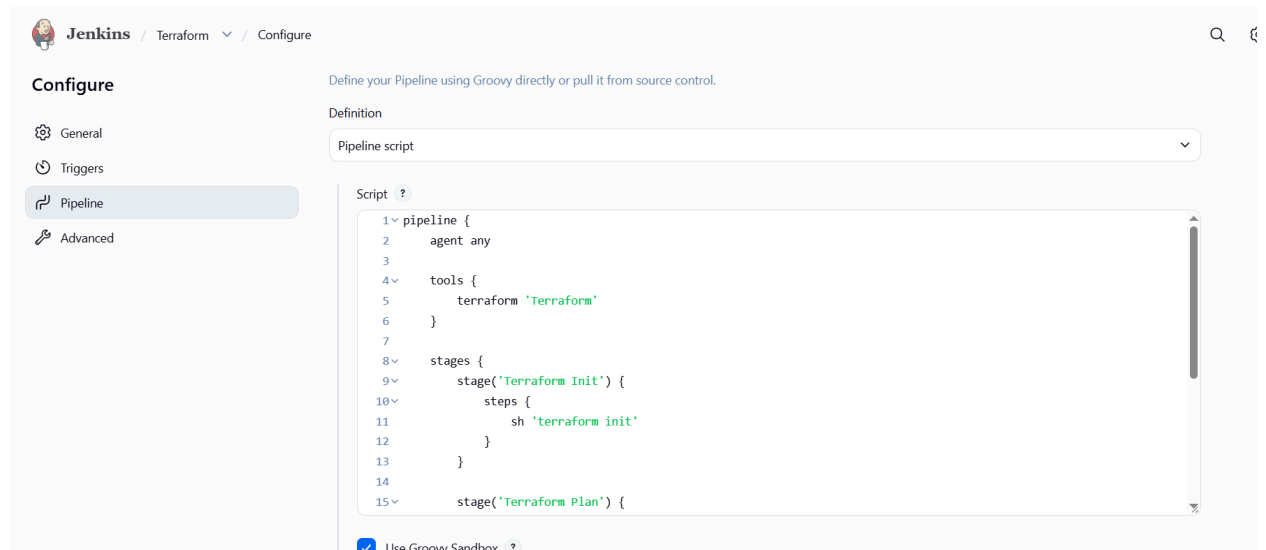
The screenshot shows the content of the Terraform state file, which is a JSON document. It contains information about the Terraform version, the configuration file, and the state of the resources.

```
terraform.tfstate > ...
1 {
2   "version": 4,
3   "terraform_version": "1.15.5",
4   "serial": 20,
5   "lineage": "e2b3056f-6bd5-8a6c-1e45-0a9ade14485c",
6   "outputs": {
7     "pet_name": {
8       "value": "MR.platypus",
9       "type": "string"
10    }
11  },
12  "resources": [
13    {
14      "mode": "managed",
15      "type": "local_file",
16      "name": "my_pet",
17      "provider": "provider[\"registry.terraform.io/hashicorp/local\"]",
18      "instances": [
19        {
20          "schema_version": 0,
21          "attributes": {
22            "content": "my cat name is MR.platypus",
23            "content_base64": null,
24            "content_base64sha256": "F4IHM/JViI8ckrlojxg1M1KZaQd0U38g/ngJPRxrEMM=",
25            "content_base64sha512": "s7jVa2r0qai7dwADNUERu+sUQfcs1etnFC8VcUumGHHiiYNSx9Qs4lBikvZJulS4KkDlB1k3WfncWVAc+FZJQ==",
26            "content_md5": "c9a3d2d6e590187b0ab5afe3fd9fb5da",
27            "content_sha1": "746db925e547a068deec611d64b7a64f1a7688fd",
28            "content_sha256": "17822133f255888f1c2ab9688f1835335299690768537f20fe71893d1c6b10c3",
29            "content_sha512": "b3b8d56b6af4a9a8bb75604074d504aeeefac5107c2b357ad35f0bc55c52e9861e122260d4b1f50b3895b88abd926e96ce0a90941d64dd67e770c54073e15925"
```

2. Integrate **Terraform** in Jenkins using the **Terraform plugin**.

```
[root@ip-172-31-14-229 ~]# cd /var/lib/jenkins/workspace/Terraform
[root@ip-172-31-14-229 Terraform]# ls
main.tf  terraform-from-jenkins.txt  terraform.tfstate
[root@ip-172-31-14-229 Terraform]# cat main.tf
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.5"
    }
  }
}

resource "local_file" "demo" {
  filename = "terraform-from-jenkins.txt"
  content  = "Terraform executed successfully from Jenkins using Terraform Plugin"
}
```



The screenshot shows the Jenkins configuration page for the Terraform plugin. The left sidebar has tabs for General, Triggers, Pipeline (selected), and Advanced. The main area is titled 'Configure' and 'Define your Pipeline using Groovy directly or pull it from source control.' Under 'Definition', 'Pipeline script' is selected. A 'Script' field contains the following Groovy code:

```
1 pipeline {
2   agent any
3
4   tools {
5     terraform 'Terraform'
6   }
7
8   stages {
9     stage('Terraform Init') {
10      steps {
11        sh 'terraform init'
12      }
13    }
14
15    stage('Terraform Plan') {
```

At the bottom, there is a checkbox labeled 'Use Groovy Sandbox' which is checked.

```
pipeline {
  agent any

  tools {
    terraform 'Terraform'
  }
```

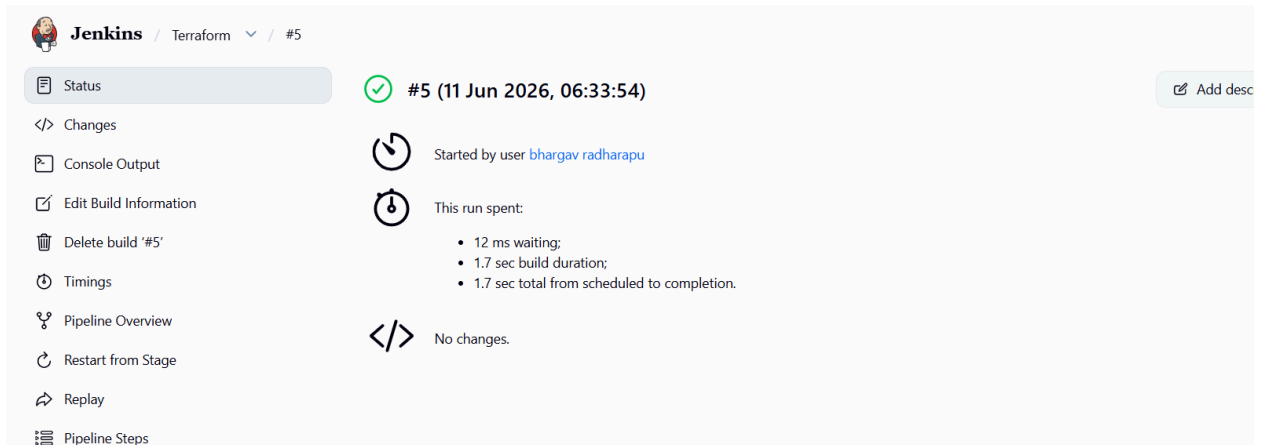
```
stages {
  stage('Terraform Init') {
    steps {
      sh 'terraform init'
    }
  }

  stage('Terraform Plan') {
    steps {
      sh 'terraform plan'
    }
  }

  stage('Terraform Apply') {
    steps {
      sh 'terraform apply -auto-approve'
    }
  }
}
```

save&build

Build successfull



The image shows the Jenkins job status page for a Terraform build. The job is named 'Terraform' and is build #5. The status is 'Success' (indicated by a green checkmark). The build was started on 11 Jun 2026 at 06:33:54. The build was started by user 'bhargav radharapu'. The build duration is 1.7 sec, and the total time from scheduled to completion is 1.7 sec. The build has no changes. The left sidebar contains links to Status, Changes, Console Output, Edit Build Information, Delete build '#5', Timings, Pipeline Overview, Restart from Stage, Replay, and Pipeline Steps.

Jenkins / Terraform / #5

Status ✓ #5 (11 Jun 2026, 06:33:54) Add desc

</> Changes

📄 Console Output

📄 Edit Build Information

🗑️ Delete build '#5'

🕒 Timings

🔗 Pipeline Overview

🔄 Restart from Stage

🔄 Replay

📋 Pipeline Steps

Started by user bhargav radharapu

This run spent:

- 12 ms waiting;
- 1.7 sec build duration;
- 1.7 sec total from scheduled to completion.

</> No changes.

Console output:



The image shows the Jenkins console output for the Terraform build. The output displays the Terraform plan and apply commands, showing that the resources were created successfully. The build finished with a SUCCESS status.

Jenkins / Terraform / #5

```
terraform {
  backend "s3" {
    bucket = "terraform-from-jenkins"
    key    = "terraform-from-jenkins.tfstate"
  }
}

resource "aws_s3_bucket" "demo" {
  bucket = "terraform-from-jenkins"
}

resource "aws_s3_bucket_policy" "demo" {
  bucket = aws_s3_bucket.demo.bucket
  policy = jsonencode({
    "Version": "2012-10-17",
    "Statement": [
      {
        "Effect": "Allow",
        "Principal": "*",
        "Action": "s3:*",
        "Resource": "*"
      }
    ]
  })
}
```

terraform plan -out=tfplan

terraform apply -auto-approve

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Finished: SUCCESS

3. Create one Jenkins job using **Maven Project** for the code below with **two stages**:
 - **Stage 1:**Git clone
 - **Stage 2:**Maven
- CompilationCode:<https://github.com/betawins/java-Working-app.git>

Install maven integration plugin:

The screenshot shows the Jenkins 'Manage Jenkins' > 'Plugins' page. A search bar at the top contains 'maven in'. The 'Available plugins' tab is selected. The table below lists several plugins:

Install	Name ↓	Released	Health
☒	Maven Integration 3.27 Build Tools This plugin provides a deep integration between Jenkins and Maven. It adds support for automatic triggers between projects depending on SNAPSHOTS as well as the automated configuration of various Jenkins publishers such as Junit.	10 mo ago	95
☐	Pipeline Maven Integration 1735.v63b_5b_1b_d25d9 pipeline Maven This plugin provides integration with Pipeline, configures maven environment to use within a pipeline job by calling sh mvn or bat mvn. The selected maven installation will be configured and prepended to the path.	5 days 20 hr ago	100
☐	Maven Info 0.3.2 List view columns Maven Adds columns configurable in views to show info about Maven jobs.	1 yr 7 mo ago	87
☐	Maven Invoker 2.5		

The screenshot shows the 'Download progress' page in Jenkins. The 'Download progress' tab is selected in the left sidebar. The progress is as follows:

- Preparation
 - Checking internet connectivity
 - Checking update center connectivity
 - Success
- Dev Tools Symbols API: Success
- Javadoc: Success
- JSch dependency: Success
- Maven Integration: Success
- Loading plugin extensions: Success

[Go back to the top page](#)

Create paven-project new item:

The screenshot shows the 'New Item' page in Jenkins. The 'All' dropdown is set to 'New Item'. The 'maven-project' name is entered in the 'Enter an item name' field. Under 'Select an item type', the 'Maven project' option is selected with a checkmark. The description for 'Maven project' is: 'Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.'

New Item

Enter an item name

maven-project

Select an item type

- ☐ Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
- ☐ Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- ☒ **Maven project**
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.
- ☐ Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
- ☐ Folder
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view,

OK

 Jenkins

maven-project

Configure



Configure

General

Source Code Management

Triggers

Environment

Pre Steps

Build

Post Steps

Build Settings

Post-build Actions

Connect and manage your code repository to automatically pull the latest code for your builds.

☐ None

☒ Git

Repositories

Repository URL

https://github.com/betawins/hiring-app.git

Credentials

- none -

+ Add

Advanced

+ Add Repository

Branches to build

Branch Specifier (blank for 'any')

*/main

+ Add Branch

Repository browser

(Auto)

Additional Behaviours

+ Add

Build

Root POM ?

pom.xml

Goals and options ?

clean install

Advanced ▾



Jenkins / maven-project

Status

Changes

Workspace

Build Now

Configure

Delete Maven project

Modules

✓ maven-project

Permalinks

- [Last build \(#1\), 2 min 50 sec ago](#)
- [Last stable build \(#1\), 2 min 50 sec ago](#)
- [Last successful build \(#1\), 2 min 50 sec ago](#)
- [Last completed build \(#1\), 2 min 50 sec ago](#)



Jenkins / maven-project ▾ / #1 ▾



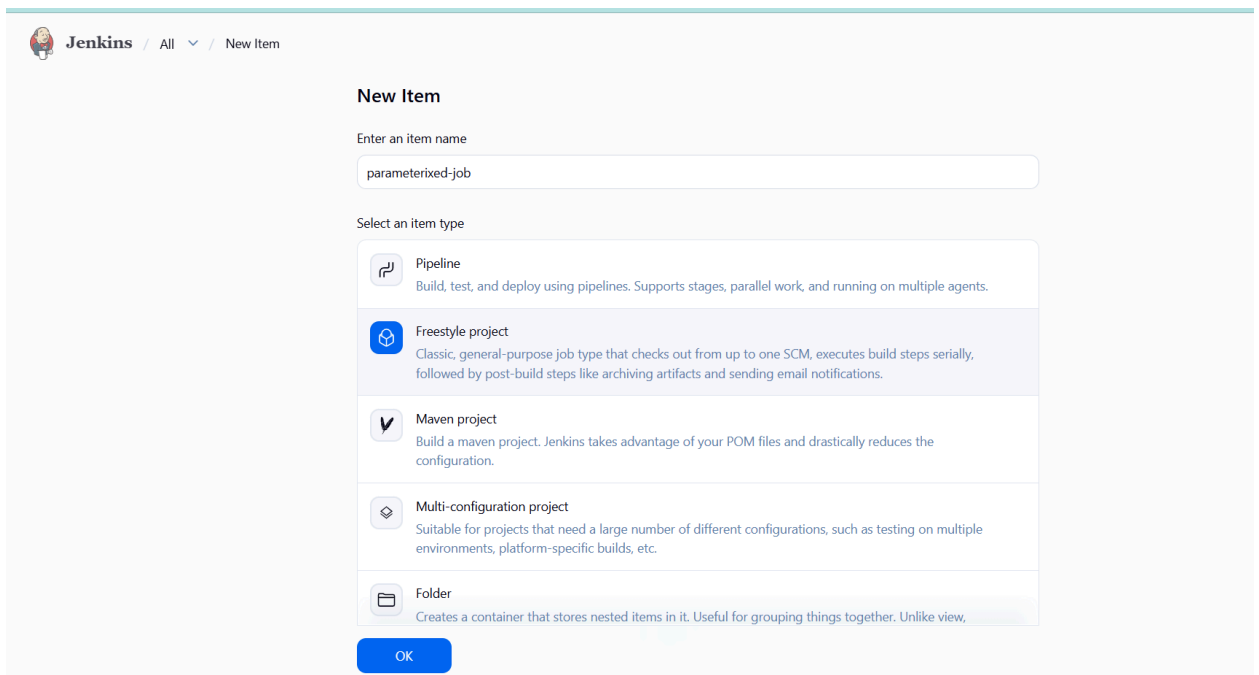
```
[INFO]
[INFO] --- war:3.3.1:war (default-war) @ hiring ---
[INFO] Packaging webapp
[INFO] Assembling webapp [hiring] in [/var/lib/jenkins/workspace/maven-project/target/hiring]
[INFO] Processing war project
[INFO] Copying webapp resources [/var/lib/jenkins/workspace/maven-project/src/main/webapp]
[INFO] Building war: /var/lib/jenkins/workspace/maven-project/target/hiring.war
[INFO]
[INFO] --- install:3.1.4:install (default-install) @ hiring ---
[INFO] Installing /var/lib/jenkins/workspace/maven-project/pom.xml to /var/lib/jenkins/.m2/repository/in/javahome/hiring/0.1/hiring-0.1.pom
[INFO] Installing /var/lib/jenkins/workspace/maven-project/target/hiring.war to /var/lib/jenkins/.m2/repository/in/javahome/hiring/0.1/hiring-0.1.war
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 4.331 s
[INFO] Finished at: 2026-06-11T10:36:32Z
[INFO] -----
Waiting for Jenkins to finish collecting data
[JENKINS] Archiving /var/lib/jenkins/workspace/maven-project/pom.xml to in.javahome/hiring/0.1/hiring-0.1.pom
[JENKINS] Archiving /var/lib/jenkins/workspace/maven-project/target/hiring.war to in.javahome/hiring/0.1/hiring-0.1.war
channel stopped
Finished: SUCCESS
```

4. Use the same code and create a **parameterized job** in Jenkins with:

- **Stage 1:**Git clone

- **Stage 2:**Maven

CompilationCode:<https://github.com/betawins/java-Worki>
[ng-app.git](https://github.com/betawins/java-Worki)



Jenkins / All / New Item

New Item

Enter an item name

parameterixed-job

Select an item type

-  Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
-  Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
-  Maven project
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.
-  Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
-  Folder
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view,

OK

Jenkins / parameterixed-job / Configure

Configure

General

Source Code Management

Triggers

Environment

Build Steps

Post-build Actions

☐ GitHub project

☒ This project is parameterised ?

String Parameter ?

Name ?

BRANCH_NAME

Default Value ?

main

Description ?

main branch

Plain text Preview

☐ Trim the string ?

Jenkins / parameterixed-job / Configure

Configure

General

Source Code Management

Triggers

Environment

Build Steps

Post-build Actions

Repository URL ?

https://github.com/betawins/hiring-app.git

Credentials ?

- none -

+ Add

Advanced

+ Add Repository

Branches to build ?

Branch Specifier (blank for 'any') ?

*/main

+ Add Branch

Jenkins / parameterixed-job

Status

Changes

Workspace

Build with Parameters

Configure

Delete Project

Rename

Project parameterixed-job

This build requires parameters:

BRANCH_NAME

main branch

main

Build Cancel

Builds >

No builds

 **Jenkins** / parameterixed-job

Status

Changes

Workspace

Build with Parameters

Configure

Delete Project

Rename

parameterixed-job

Permalinks

- Last build (#1), 30 sec ago
- Last stable build (#1), 30 sec ago
- Last successful build (#1), 30 sec ago
- Last completed build (#1), 30 sec ago

Builds >

Filter

Today

- #1 10:46

 **Jenkins** / parameterixed-job / #1

Status

Changes

Console Output

Edit Build Information

Delete build '#1'

Parameters

Timings

Git Build Data

Console

Download

Copy

View as plain text

```
Started by user bhargav radharapu
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/parameterixed-job
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/betawins/hiring-app.git
> git init /var/lib/jenkins/workspace/parameterixed-job # timeout=10
Fetching upstream changes from https://github.com/betawins/hiring-app.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/betawins/hiring-app.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git config remote.origin.url https://github.com/betawins/hiring-app.git # timeout=10
> git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 9405b107fdffdf68e69fe953eaf9669784f9d6a (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 9405b107fdffdf68e69fe953eaf9669784f9d6a # timeout=10
Commit message: "Update README.md"
First time build. Skipping changelog.
Finished: SUCCESS
```

5. What are the **global variables** in Jenkins?

Global variables in Jenkins are predefined environment variables accessible

across jobs and pipelines. They provide build-related information such as

build number, workspace path, job name, Git branch, and Jenkins home

directory.

Some commonly used global variables include

`BUILD_NUMBER`, which stores the current build number,

`JOB_NAME`, which contains the name of the Jenkins job, and

`WORKSPACE`, which defines the location where Jenkins clones the source code

and executes builds.

Variables like `GIT_COMMIT` and `GIT_BRANCH` help identify the exact Git

commit and branch used during the build process.

These global variables are useful in automation, scripting, and CI/CD pipelines

to make Jenkins jobs more dynamic and reusable.